This chapter introduces the fundamental concepts of program analysis, exploring how we can mathematically assign meaning to syntax and systematically reason about program behaviors.

---

1. Overview and Motivation

Program analysis is the systematic examination of a program to determine its properties. We apply it for various reasons:

- Finding bugs (historical fun fact: the first "bug" was an actual moth found in the Harvard Mark II in 1947).
- Optimizing performance and building compilers.
- Detecting security vulnerabilities and improving software maintainability.

To check if a program behaves as intended, we must verify if the code satisfies a specific semantic property. However, this is fundamentally constrained by **Rice's Theorem**: there exists no algorithm that can automatically, universally, and exactly determine if a program satisfies a non-trivial semantic property.

Because we cannot have a tool that is simultaneously automatic, universal, and exact, we must rely on different techniques that trade off these traits:

- **Testing:** Unsound (can accept incorrect programs), but complete (does not reject safe programs).
- **Machine-Assisted Proving (e.g., Coq, Isabelle):** Sound and quasi-complete, but requires manual effort to supply invariants.
- **Finite-State Model Checking:** Automatic, sound, and complete, but only with respect to finite models (can fail to terminate on infinite states).
- **Conservative Static Analysis:** Automatic and sound, but incomplete (relies on over-approximation).

---

2. Formal Semantics and State

Semantics assigns mathematical meaning to syntax. We distinguish between the syntactic representation of a program (how it is written, c) and its computed function ($[[c]]$).

- **Memory State:** A state σ is a function mapping variables to integers: $\sigma : \mathbb{X} \to \mathbb{Z}$.
- **State Space:** The set of all possible states is denoted as $\Sigma \triangleq \{\sigma : \mathbb{X} \to \mathbb{Z}\}$.
- **State Update:** The notation $\sigma[n/\mathtt{x}]$ or $\sigma[\mathtt{x} \mapsto n]$ denotes a state identical to σ, except the variable $\mathtt{x}$ evaluates to n.

2.1 Forward Semantics

For deterministic programs, the denotational semantics models a program as a function from an input state to an output state: $[[c]] : \Sigma \to \Sigma_\perp$.

- The set $\Sigma_\perp = \Sigma \uplus \{\perp\}$ includes a special symbol $\perp$ to represent non-terminating executions.

2.2 Collecting Semantics

To consider all possible execution trajectories simultaneously (especially useful for non-deterministic programs or abstraction), we lift the semantics to operate on sets of states (the powerset $\wp(\Sigma)$).

- **Collecting Semantics:** $[[c]] : \wp(\Sigma) \to \wp(\Sigma)$.
- Defined as: $[[c]]P = \bigcup_{\sigma \in P} [[c]]\sigma$.

---

3. Approximations

Since exact analysis is impossible due to undecidability, we use approximations.

- **Over-approximations:** Expand the set of possible behaviors. If the over-approximated program doesn't reach a bad state, the exact program won't either.
  - *Pro:* Good for proving correctness (Sound).
  - *Con:* Can report bugs that don't exist (False Positives).
- **Under-approximations:** Restrict the set of possible behaviors. If the under-approximated program reaches a bad state, the exact program definitely will.
  - *Pro:* Good for bug-finding.
  - *Con:* Can miss actual bugs (False Negatives).

---

4. Denotational Semantics of IMP

We construct the meaning of a program compositionally.

4.1 Expressions

- **Arithmetic Expressions (Aexp):** $[[\cdot]] : \text{Aexp} \to \Sigma \to \mathbb{Z}$. Example: $[[\mathtt{a}_0 \text{ op } \mathtt{a}_1]]\sigma \triangleq [[\mathtt{a}_0]]\sigma \text{ op } [[\mathtt{a}_1]]\sigma$.
- **Boolean Expressions (Bexp):** In collecting semantics, a boolean expression filters a set of states, keeping only those where the condition holds: $[[\cdot]] : \text{Bexp} \to \wp(\Sigma) \to \wp(\Sigma)$. Example: $[[\mathtt{a}_0 \text{ cmp } \mathtt{a}_1]]P \triangleq \{\sigma \in P \mid [[\mathtt{a}_0]]\sigma \text{ cmp } [[\mathtt{a}_1]]\sigma\}$.

4.2 Commands and Control Flow

Command semantics $[[\cdot]] : \text{Com} \to \wp(\Sigma) \to \wp(\Sigma)$ are defined recursively:

- **Skip:** $[[\mathtt{skip}]]P \triangleq P$.
- **Assignment:** $[[\mathtt{x} := a]]P \triangleq \{\sigma[n/\mathtt{x}] \mid \sigma \in P, n = [[a]]\sigma\}$.
- **Sequence:** $[[c_1; c_2]]P \triangleq [[c_2]]([[c_1]]P)$.
- **Choice (Non-determinism):** $[[c_1 + c_2]]P \triangleq [[c_1]]P \cup [[c_2]]P$.

4.3 Loops and Fixpoints

Loops are handled using the **Kleene Star** operator, which applies a command zero or more times: $[[c^\star]]P \triangleq \bigcup_{k=0}^{\infty}[[c]]^k P$. To encode a `while` loop, we apply the loop body c as long as condition b holds, and finally filter by $\neg b$:

$$[[\mathtt{while\ b\ do\ c}]]P \triangleq [[\neg b]] \bigcup_{k=0}^{\infty} ([[c]] \circ [[b]])^k P$$

This represents finding the Least Fixed Point of the semantic function mapping states through iterations of the loop.

---

5. Correctness and Preconditions

We use semantic definitions to formally prove if a program starting in a set of input states P will only result in valid output states Q.

- **Partial Correctness:** Denoted as $[[c]]P \subseteq Q$. This means that starting from P, the execution of c either does not terminate, or terminates in a state satisfying Q.

To systematically calculate correctness, we compute preconditions:

5.1 Dijkstra's Weakest Liberal Precondition (wlp)

Calculates the largest set of input states from which execution *either* diverges *or* ends in Q:

$$wlp(c, Q) = \{\sigma \mid [[c]]\{\sigma\} \subseteq Q\}$$

Thus, $[[c]]P \subseteq Q$ if and only if $P \subseteq wlp(c, Q)$.

5.2 Backward Semantics and Weakest Possible Precondition (wpp)

Backward semantics reverses the relation: $[[c]]_{op} \triangleq \{(\delta, \sigma) \mid (\sigma, \delta) \in [[c]]\}$. Hoare's Weakest Possible Precondition calculates the largest set of input states that have *at least one successful computation* reaching Q:

$$wpp(c, Q) = \{\sigma \mid [[c]]\sigma \cap Q \neq \emptyset\}$$

Conveniently, this maps directly to the backward semantics: $wpp(c, Q) = [[c]]_{op}Q$.

02. Denotational Semantics

This chapter formalizes how to assign mathematical meaning to programs, mapping syntax to its computed function. It focuses on collecting semantics, which is the foundation for defining approximations in static analysis.

---

1. Preliminaries and State

Semantics assigns meaning to syntax. Given a program written in a specific syntax c, its meaning is denoted as $[[c]]$.

- **State (σ):** A concrete memory state is a function mapping variables to integers, $\sigma : \mathbb{X} \to \mathbb{Z}$.
- **State Space (Σ):** The set of all possible states is $\Sigma \triangleq \{\sigma : \mathbb{X} \to \mathbb{Z}\}$.
- **State Update:** The notation $\sigma[n/\mathtt{x}]$ or $\sigma[\mathtt{x} \mapsto n]$ indicates a state identical to σ, except the variable $\mathtt{x}$ has the value n.
- **Concrete Domain:** In program analysis, we often work with sets of states. The concrete domain is the powerset of Σ, denoted as $\wp(\Sigma) \triangleq \{P \mid P \subseteq \Sigma\}$.

---

2. Semantics of Expressions

Expressions are evaluated relative to a given state or set of states.

2.1 Arithmetic Expressions (`Aexp`)

The denotational semantics of an arithmetic expression evaluates to an integer given a specific state: $[[\cdot]] : \text{Aexp} \to \Sigma \to \mathbb{Z}$.

- $[[n]]\sigma \triangleq n$
- $[[\mathtt{x}]]\sigma \triangleq \sigma(\mathtt{x})$
- $[[\mathtt{a}_0 \text{ op } \mathtt{a}_1]]\sigma \triangleq [[\mathtt{a}_0]]\sigma \text{ op } [[\mathtt{a}_1]]\sigma$

2.2 Boolean Expressions (`Bexp`)

In collecting semantics, a boolean expression is treated as a filter over a set of states. It returns the subset of states that satisfy the condition: $[[\cdot]] : \text{Bexp} \to \wp(\Sigma) \to \wp(\Sigma)$.

- $[[\text{true}]]P \triangleq P$
- $[[\text{false}]]P \triangleq \emptyset$
- $[[\mathtt{a}_0 \text{ cmp } \mathtt{a}_1]]P \triangleq \{\sigma \in P \mid [[\mathtt{a}_0]]\sigma \text{ cmp } [[\mathtt{a}_1]]\sigma\}$
- $[[\text{not } b]]P \triangleq P \setminus ([[b]]P)$

Note: $[[b]]P \subseteq P$ always holds.

---

3. Regular Commands and Collecting Semantics

For deterministic programs, semantics can be defined as $[[c]] : \Sigma \to \Sigma_\perp$, where $\Sigma_\perp = \Sigma \uplus \{\perp\}$ handles non-termination. However, to handle non-determinism and lay the groundwork for abstraction, we use **Collecting Semantics**, which maps a set of input states to a set of output states: $[[c]] : \wp(\Sigma) \to \wp(\Sigma)$.

By definition, $[[c]]P = \bigcup_{\sigma \in P} [[c]]\{\sigma\}$.

3.1 Base Commands

- **Skip:** $[[\mathtt{skip}]]P \triangleq P$
- **Assignment:** $[[\mathtt{x} := a]]P \triangleq \{\sigma[n/\mathtt{x}] \mid \sigma \in P, n = [[a]]\sigma\}$
- **Guard/Filter:** $[[b?]]P \triangleq [[b]]P$
- **Sequence:** $[[c_1; c_2]]P \triangleq [[c_2]]([[c_1]]P)$
- **Non-deterministic Choice:** $[[c_1 + c_2]]P \triangleq [[c_1]]P \cup [[c_2]]P$

3.2 Loops and Kleene Star

To execute a command an arbitrary number of times, we use the Kleene star operator ($c^\star$).

- $[[c^\star]]P \triangleq \bigcup_{k=0}^{\infty} [[c]]^k P$

This is mathematically resolved using Kleene's fixpoint theorem. The semantics of $[[c^\star]]P$ represents the smallest set of states that contains the initial states P and is closed under the execution of c: $[[c^\star]]P = P \cup [[c]]([[c^\star]]P)$

3.3 Encoding High-Level Control Flow

Standard control flow constructs are encoded using regular commands:

- **If-Then-Else:** $\mathtt{if\ b\ then\ c_1\ else\ c_2} \triangleq (b?; c_1) + (\neg b?; c_2)$
  - Semantics: $[[\mathtt{if\ b\ then\ c_1\ else\ c_2}]]P \triangleq [[c_1]]([[b]]P) \cup [[c_2]]([[\neg b]]P)$

- **While Loop:** `while b do c` $\triangleq (b?; c)^\star; \neg b?$
  - Semantics: $[[\texttt{while b do c}]]P \triangleq [[\neg b]] \left(\bigcup_{k=0}^{\infty}([[c]] \circ [[b]])^k P\right)$

---

4. Partial Correctness and Preconditions

We formalize correctness by checking if the reachable states conform to a specification.

Partial Correctness: Given a set of input states P and admissible outputs Q, a program c is partially correct if $[[c]]P \subseteq Q$. This means execution either diverges (does not terminate) or reaches a state in Q.

4.1 Dijkstra's Weakest Liberal Precondition (wlp)

The Weakest Liberal Precondition calculates the largest set of input states from which all terminating computations are guaranteed to end in Q.

$$wlp(c, Q) = \{\sigma \mid [[c]]\{\sigma\} \subseteq Q\}$$

Theorem: $[[c]]P \subseteq Q \iff P \subseteq wlp(c, Q)$.

4.2 Relational and Backward Semantics

Collecting semantics can be equivalently expressed as a relation $[[c]] \subseteq \Sigma \times \Sigma$, where $(\sigma, \delta) \in [[c]]$ means that starting from σ, state δ is reachable.

Backward Semantics: The opposite relation maps output states back to their possible input states.

$$[[c]]_{op} \triangleq \{(\delta, \sigma) \mid (\sigma, \delta) \in [[c]]\}$$

For example, $[[c_1; c_2]]_{op} \triangleq [[c_1]]_{op} \circ [[c_2]]_{op}$.

4.3 Hoare's Weakest Possible Precondition (wpp)

The Weakest Possible Precondition identifies the largest set of input states that have *at least one* successful computation ending in Q.

$$wpp(c, Q) = \{\sigma \mid [[c]]\{\sigma\} \cap Q \neq \emptyset\}$$

This directly corresponds to applying the backward semantics to the set of target states:

$wpp(c, Q) = [[c]]_{op} Q$.

03. Total Correctness

This chapter extends the reasoning of Hoare Logic to ensure **termination**. While partial correctness only guarantees the result *if* the program ends, **Total Correctness** ensures that the program *must* end and satisfy the postcondition.

---

1. Defining Total Correctness

A total correctness assertion is written using square brackets to distinguish it from partial correctness:

$$[P]\ c\ [Q]$$

1.1 Formal Definition

The triple $[P]\ c\ [Q]$ is valid (written $\models [P]\ c\ [Q]$) if and only if for every initial state σ such that $\sigma \models P$:

1. The execution of command c starting from σ **must terminate**.
2. The final state σ' **must satisfy** the postcondition Q ($\sigma' \models Q$).

1.2 Relation to Partial Correctness

Total correctness is the conjunction of partial correctness and termination:

$$[P]\ c\ [Q] \iff \{P\}\ c\ \{Q\} \wedge (\text{termination of } c \text{ from } P)$$

Consequently, proving total correctness is strictly harder than proving partial correctness.

---

2. Mathematical Foundations: Well-Founded Relations

To prove termination, especially for loops, we need a way to show that a program cannot run forever. We do this by mapping program states to a set that cannot have infinite decreasing sequences.

2.1 Well-Founded Relations

Definition: A binary relation $\prec$ on a set A is **well-founded** if there are no infinite descending chains $a_0 \succ a_1 \succ a_2 \succ \ldots$ in A.

- **Example (Well-founded):** The set of natural numbers with the standard "less than" relation $(\mathbb{N}, <)$. Any sequence of decreasing natural numbers must eventually hit zero.
- **Example (Not well-founded):** The set of integers $(\mathbb{Z}, <)$. One can decrease indefinitely into negative numbers $(0 > -1 > -2 \ldots)$.

2.2 Minimal Element Principle

A relation $\prec$ is well-founded if and only if every non-empty subset $Q \subseteq A$ has at least one **minimal element** m (an element such that no $x \in Q$ satisfies $x \prec m$).

---

3. Inference Rules for Total Correctness

Most rules for total correctness (Skip, Assignment, Sequence, Conditional) are identical to those in partial correctness because these constructs naturally terminate if their components do. The primary change is the **While Rule**.

3.1 The Total While Rule

To prove $[P]$ `while` b `do` c $[P \wedge \neg b]$, we introduce a **Variant** (also called a ranking function) t.

$$\frac{\forall z \in A.\ [P \wedge b \wedge t = z]\ c\ [P \wedge t \prec z]}{\vdash [P]\ \texttt{while}\ b\ \texttt{do}\ c\ [P \wedge \neg b]}$$

Requirements for the Rule:

- **Invariant:** P must be a valid loop invariant.
- **Well-founded Set:** $(A, \prec)$ must be a well-founded relation.
- **Variant Decrease:** In each iteration, the value of the expression t must strictly decrease according to $\prec$.

Because t decreases in a well-founded set, it cannot decrease forever; therefore, the loop must eventually terminate.

---

4. Practical Example: Counting Down

Consider a simple loop that decrements a counter:

$$\texttt{while}\ (x > 0)\ \texttt{do}\ x := x - 1$$

1. **Invariant (P):** $x \geq 0$.
2. **Variant (t):** The value of x itself.
3. **Well-founded Relation:** $(\mathbb{N}, <)$.
4. **Proof Step:** We must show that if the loop executes $(x > 0)$, the new value x' satisfies $x' < x$. Since $x' = x - 1$, this holds for all $x > 0$.

---

5. Weakest Preconditions (wp)

In total correctness, Dijkstra's **Weakest Precondition** (wp) specifically requires termination, unlike the Weakest Liberal Precondition (wlp).

- **Partial Correctness (wlp):** $wlp(c, Q) = \{\sigma \mid [[c]]\sigma \subseteq Q\}$.
- **Total Correctness (wp):** $wp(c, Q) = \{\sigma \mid [[c]]\sigma \neq \emptyset \wedge [[c]]\sigma \subseteq Q\}$.

The condition $[[c]]\sigma \neq \emptyset$ explicitly forces the existence of a final state, thereby ensuring termination.

04. Hoare Logic

This chapter introduces **Axiomatic Semantics** via **Hoare Logic**, a formal system used to reason about the correctness of computer programs by using logical triples.

---

1. Hoare Triples

The fundamental unit of reasoning in Hoare Logic is the **Hoare Triple**, which relates a program's behavior to logical assertions about the state before and after its execution.

A triple is written as:

$$\{P\}\ c\ \{Q\}$$

- P **(Precondition):** An assertion that must hold before the execution of command c.
- c **(Command):** The program or fragment being analyzed.
- Q **(Postcondition):** An assertion that is guaranteed to hold after c finishes execution, provided P was initially true.

1.1 Partial Correctness

Hoare Logic primarily deals with **Partial Correctness**. A triple $\{P\}\ c\ \{Q\}$ is valid (written $\models \{P\}\ c\ \{Q\}$) if:

> For every initial state σ that satisfies P, if the execution of c starting from σ terminates in a final state σ', then σ' must satisfy Q.

Crucially, partial correctness does not require the program to terminate. If the program loops forever, the triple is vacuously true.

---

2. Inference Rules for Partial Correctness

To prove a triple is valid, we use a set of **Inference Rules** to derive it formally (written $\vdash \{P\}\ c\ \{Q\}$).

2.1 Basic Rules

- **Skip Rule:** The state does not change, so the assertion remains the same.

$$\frac{}{\{P\}\ \texttt{skip}\ \{P\}}$$

- **Assignment Rule:** To ensure P holds after $x := a$, it must hold for the expression a beforehand. $P[a/x]$ denotes the substitution of x with a in P.

$$\frac{}{\{P[a/x]\}\ x := a\ \{P\}}$$

2.2 Structural Rules

- **Sequence Rule:** To reason about $c_1; c_2$, we find an intermediate assertion R.

$$\frac{\{P\}\ c_1\ \{R\} \quad \{R\}\ c_2\ \{Q\}}{\{P\}\ c_1; c_2\ \{Q\}}$$

- **Conditional Rule:** We analyze both branches of an `if` statement.

$$\frac{\{P \wedge b\}\ c_1\ \{Q\} \quad \{P \wedge \neg b\}\ c_2\ \{Q\}}{\{P\}\ \texttt{if}\ b\ \texttt{then}\ c_1\ \texttt{else}\ c_2\ \{Q\}}$$

2.3 The While Rule and Invariants

Reasoning about loops requires a **Loop Invariant** P, a property that holds before, during, and after every iteration of the loop.

$$\frac{\{P \wedge b\}\ c\ \{P\}}{\{P\}\ \texttt{while}\ b\ \texttt{do}\ c\ \{P \wedge \neg b\}}$$

- If the invariant P is preserved by the body c when b is true, it will still hold when the loop eventually terminates (at which point b is false).

2.4 Rule of Consequence

This rule allows us to strengthen preconditions or weaken postconditions using standard logic.

$$\frac{P' \implies P \quad \{P\}\ c\ \{Q\} \quad Q \implies Q'}{\{P'\}\ c\ \{Q'\}}$$

---

3. Validity vs. Derivability

There is a distinction between a property being "true" in the model and being "provable" in the logic:

- **Validity ($\models \{P\}\ c\ \{Q\}$):** The property holds according to the denotational/operational semantics.
- **Derivability ($\vdash \{P\}\ c\ \{Q\}$):** The property can be proven using the inference rules listed above.

3.1 Soundness and Completeness

- **Soundness:** Every triple we can derive is actually valid.

$$\vdash \{P\}\ c\ \{Q\} \implies \models \{P\}\ c\ \{Q\}$$

- **Relative Completeness:** Every valid triple can be derived, assuming we have a perfect oracle for the underlying mathematical logic (e.g., arithmetic).

$$\models \{P\}\ c\ \{Q\} \implies \vdash \{P\}\ c\ \{Q\}$$

---

4. Practical Application: Verification Conditions

In practice, human experts or automated tools use **Weakest Preconditions (wp)** to generate **Verification Conditions (VCs)**.

1. Start with the desired postcondition Q.
2. Work backward through the program to find the weakest possible precondition.
3. Check if the given precondition P implies this weakest precondition.

Rule	Backward Step
Assignment	$wp(x := a, Q) = Q[a/x]$
Sequence	$wp(c_1; c_2, Q) = wp(c_1, wp(c_2, Q))$

05. Necessary Condition Logic

This chapter introduces **Necessary Condition Logic (NC Logic)**, which shifts the perspective from forward execution to backward reasoning using **over-approximation**. While Hoare Logic asks "what is guaranteed to happen if we start in P?", NC Logic asks "what *must* have been true at the start if we eventually reached Q?".

---

1. The Core Concept: Backward Over-Approximation

NC Logic is the backward counterpart to Hoare Logic. It is used to find the **necessary conditions** for a specific outcome (e.g., reaching a target state, a bug, or a specific line of code).

1.1 Semantic Definition

We denote an NC triple using angle brackets:

$$\langle P \rangle \; c \; \langle Q \rangle$$

This triple is valid if and only if:

$$[[c]]_{op} Q \subseteq P$$

Meaning: The set of all initial states that can possibly lead to a state in Q via command c ($[[c]]_{op}Q$) is a subset of P. Therefore, starting in P is a *necessary condition* to reach Q. If the initial state was not in P, it is impossible to reach Q.

1.2 Comparison with Hoare Logic

- **Hoare Logic (HL):** $\{P\} \; c \; \{Q\} \iff [[c]]P \subseteq Q$ (Forward Over-approximation).
- **NC Logic:** $\langle P \rangle \; c \; \langle Q \rangle \iff [[c]]_{op}Q \subseteq P$ (Backward Over-approximation).

By expanding the definition, $\langle P \rangle \; c \; \langle Q \rangle$ is equivalent to saying: $\forall \sigma, \sigma'. \; (\sigma, \sigma') \in [[c]] \wedge \sigma' \in Q \implies \sigma \in P$.

---

2. Inference Rules for NC Logic

The inference rules for NC Logic are structurally similar to Hoare Logic but are designed to propagate the post-condition Q backwards to find the necessary pre-condition P.

2.1 Basic Rules

- **Skip:**

$$\frac{}{\langle P \rangle \; \texttt{skip} \; \langle P \rangle}$$

- **Assignment:** The exact set of states that lead to Q after $x := e$ is $Q[e/x]$. Since we are over-approximating, any P that contains $Q[e/x]$ is a valid necessary condition.

$$\frac{}{\langle Q[e/x] \rangle \; x := e \; \langle Q \rangle}$$

2.2 Structural Rules

- **Sequence:**

$$\frac{\langle P \rangle \; c_1 \; \langle R \rangle \quad \langle R \rangle \; c_2 \; \langle Q \rangle}{\langle P \rangle \; c_1; c_2 \; \langle Q \rangle}$$

- **Choice (Non-determinism/If-statements):** If Q can be reached via either branch, the necessary condition must account for both possibilities. Thus, we take the union.

$$\frac{\langle P_1 \rangle\ c_1\ \langle Q \rangle \quad \langle P_2 \rangle\ c_2\ \langle Q \rangle}{\langle P_1 \cup P_2 \rangle\ c_1 + c_2\ \langle Q \rangle}$$

2.3 Rule of Consequence

In NC Logic, since P is an over-approximation of the pre-states, we can safely **weaken the precondition** (make it larger) or **strengthen the postcondition** (make it smaller).

$$\frac{P \implies P' \quad \langle P \rangle\ c\ \langle Q \rangle \quad Q' \implies Q}{\langle P' \rangle\ c\ \langle Q' \rangle}$$

(Note how this is the exact opposite of the Hoare Logic consequence rule).

---

3. Applications

NC Logic is extremely useful in **security analysis and debugging**. If Q represents an exploitation state (e.g., a buffer overflow or unauthorized access), computing $\langle P \rangle\ c\ \langle Q \rangle$ gives us P: the conditions that an attacker *must* satisfy to trigger the exploit. If P evaluates to `false`, the bug is unreachable.

06. Incorrectness Logic

This chapter introduces **Incorrectness Logic (IL)**, a formal system designed for **bug-finding** and proving the **presence of errors** rather than their absence. Unlike Hoare Logic, which uses over-approximation to prove correctness, IL uses **under-approximation** to ensure that every reported bug is a real one (No False Positives).

---

1. Motivation: From Verification to Bug-Finding

While Hoare Logic aims to prove that no bad states are reachable, Incorrectness Logic focuses on proving that a specific bad state *is* reachable.

- **Hoare Logic (Verification):** Focuses on "any accepted program satisfies the property" (Soundness). It is prone to False Positives due to over-approximation.
- **Incorrectness Logic (Bug-Finding):** Focuses on "any reported bug is a real bug". It uses under-approximation to avoid False Positives, though it may have False Negatives (missing some bugs).

---

2. The Incorrectness Triple

The core construct is the **Incorrectness Triple**, which uses square brackets but with a different semantic meaning than Hoare's total correctness:

$$[P]\ c\ [Q]$$

2.1 Semantic Definition

An incorrectness triple $[P]\ c\ [Q]$ is valid if and only if:

$$Q \subseteq [[c]]P$$

This means that **every state** in the post-condition Q is **reachable** from at least one state in the pre-condition P after executing command c.

2.2 Comparison of Triples

- **Hoare (Over-approximation):** $\{P\}\ c\ \{Q\} \iff [[c]]P \subseteq Q$.
- **Incorrectness (Under-approximation):** $[P]\ c\ [Q] \iff Q \subseteq [[c]]P$.

---

3. Inference Rules for Incorrectness Logic

The rules for IL are designed to propagate reachability information forward.

3.1 Basic Rules

- **Assignment Rule:** A state reached after an assignment is reachable if the original state was in P.

$$\frac{}{[P]\ x := e\ [\{\sigma[[[e]]\sigma/x] \mid \sigma \in P\}]}$$

- **Sequence Rule:** Reachability is transitive through sequential execution.

$$\frac{[P]\ c_1\ [R] \quad [R]\ c_2\ [Q]}{[P]\ c_1; c_2\ [Q]}$$

3.2 Choice and Iteration

- **Choice Rule:** Since IL tracks what *can* happen, the choice rule takes the union of reachable states from both branches.

$$\frac{[P]\ c_1\ [Q_1] \quad [P]\ c_2\ [Q_2]}{[P]\ c_1 + c_2\ [Q_1 \cup Q_2]}$$

- **Iteration (Unrolling):** To find bugs in loops, IL typically explores the loop for a finite number of iterations (k).

$$\frac{[P]\ c^k\ [Q]}{[P]\ c^*\ [Q]}$$

  If a state is reachable in exactly k steps, it is reachable in the loop.

---

4. Handling Errors: Real Incorrectness Logic

In real-world analysis, we distinguish between normal termination and error states.

4.1 Exit Conditions

An extended triple tracks the "exit status" ϵ (e.g., `ok` for normal termination, `er` for an error/bug):

$$[P]\ c\ [\epsilon : Q]$$

- **Reachability of Errors:** $[P]\ c\ [\text{er} : Q]$ means every state in Q is an error state reachable from P.
- **Short-circuiting:** If a command c_1 results in an error, the subsequent command c_2 in a sequence $c_1; c_2$ is not executed for that specific trajectory.

4.2 Application to Static Analysis

IL is the foundation for modern "bug-hunting" tools (like **Facebook Infer** or **Meta Zoncolan**). These tools prioritize reporting "True Positives" to avoid "alarm fatigue" among developers.

04. Sufficient Incorrectness Logic

This chapter introduces **Sufficient Incorrectness Logic (SIL)**. While standard Incorrectness Logic (IL) uses forward under-approximation, SIL uses **backward under-approximation**. It is designed to find conditions that are strictly **sufficient** to trigger a specific state or bug.

---

1. The Core Concept: Backward Under-Approximation

SIL is the backward counterpart to Incorrectness Logic. It answers the question: "What starting conditions are *sufficient* to guarantee that a path to Q exists?".

1.1 Semantic Definition

We denote a SIL triple using floor brackets (or sometimes a variant of standard brackets depending on the literature convention):

$$\lfloor P \rfloor\ c\ \lfloor Q \rfloor$$

This triple is valid if and only if:

$$P \subseteq [[c]]_{op} Q$$

Meaning: Every state in P has at least one valid execution path through c that terminates in Q. Therefore, satisfying P is *sufficient* to reach Q.

1.2 Symmetry of the Four Logics

To see the complete picture of Program Analysis logics: 1. **Hoare Logic (HL):** $[[c]]P \subseteq Q$ (Forward Over) - *Absence of bugs.* 2. **Incorrectness Logic (IL):** $Q \subseteq [[c]]P$ (Forward Under) - *Presence of bugs from P.* 3. **Necessary Condition (NC):** $[[c]]_{op} Q \subseteq P$ (Backward Over) - *Required conditions to reach Q.* 4. **Sufficient Incorrectness (SIL):** $P \subseteq [[c]]_{op} Q$ (Backward Under) - *Sufficient conditions to reach Q.*

---

2. Inference Rules for SIL

The rules for SIL mirror those of IL but operate in reverse, propagating the post-condition Q backwards to find a sufficient pre-condition P.

2.1 Basic Rules

- **Skip:**

$$\frac{}{\lfloor P \rfloor \ \texttt{skip} \ \lfloor P \rfloor}$$

- **Assignment:** Since $Q[e/x]$ is the exact set of pre-states, any subset P is a sufficient condition.

$$\frac{}{\lfloor Q[e/x] \rfloor \ x := e \ \lfloor Q \rfloor}$$

2.2 Structural Rules

- **Sequence:**

$$\frac{\lfloor P \rfloor \ c_1 \ \lfloor R \rfloor \quad \lfloor R \rfloor \ c_2 \ \lfloor Q \rfloor}{\lfloor P \rfloor \ c_1; c_2 \ \lfloor Q \rfloor}$$

- **Choice (Non-determinism):** Unlike NC Logic which unions preconditions, in SIL, if P_1 is sufficient to reach Q through c_1, then P_1 is already sufficient for the entire choice construct.

$$\frac{\lfloor P_1 \rfloor \ c_1 \ \lfloor Q \rfloor}{\lfloor P_1 \rfloor \ c_1 + c_2 \ \lfloor Q \rfloor} \quad \text{or} \quad \frac{\lfloor P_2 \rfloor \ c_2 \ \lfloor Q \rfloor}{\lfloor P_2 \rfloor \ c_1 + c_2 \ \lfloor Q \rfloor}$$

2.3 Rule of Consequence

Since SIL tracks an under-approximation (P is a subset of the actual pre-states), we can safely **strengthen the precondition** (make it smaller) or **weaken the postcondition** (make it larger).

$$\frac{P' \implies P \quad \lfloor P \rfloor \ c \ \lfloor Q \rfloor \quad Q \implies Q'}{\lfloor P' \rfloor \ c \ \lfloor Q' \rfloor}$$

---

3. Applications

SIL is the ultimate tool for **automated exploit generation and test-case generation**. If Q is an assertion failure or a crash, finding a valid SIL triple $\lfloor P \rfloor \ c \ \lfloor Q \rfloor$ means that *any* state satisfying P will definitively trigger the bug. You can plug the constraints of P into an SMT solver (like Z3) to generate a concrete test case that is mathematically guaranteed to crash the program.